

# Cadence

iOS App — Code Review

Reviewed May 26, 2026 · 29 Swift files · SwiftUI + SwiftData

## Overview

Cadence is a well-structured health journaling app. The architecture is clean — MVVM with a clear boundary between Models, Services, and Features, consistent **@MainActor** usage, and a sensible SwiftData schema. The issues below are correctness bugs and design gaps, not style problems. High-priority items involve silent failures and broken notification logic; lower-priority items are code-quality improvements.

## Summary

| #  | Issue                                                    | Priority |
|----|----------------------------------------------------------|----------|
| 1  | Streak notification never fires when today is incomplete | High     |
| 2  | stressFatiguePattern checks wrong log index (off-by-one) | High     |
| 3  | Silent save failures give false success feedback         | High     |
| 4  | analyze() stub is dead code — misleading                 | Medium   |
| 5  | Streak-at-risk notification fires wrong day after 9 PM   | Medium   |
| 6  | Default sleepHours = 7.0 skews sleep correlation         | Medium   |
| 7  | InsightCard.id is var instead of let                     | Low      |
| 8  | Hardcoded magic confidence scores (0.85, 0.78)           | Low      |
| 9  | Force unwraps in calendar math                           | Low      |
| 10 | Synchronous pattern computation on @MainActor            | Low      |
| 11 | PDF export overwrites previous file (fixed filename)     | Low      |

## Detailed Findings

### Bugs

|        |                                                          |      |
|--------|----------------------------------------------------------|------|
| #<br>1 | Streak notification never fires when today is incomplete | High |
|--------|----------------------------------------------------------|------|

Features/Dashboard/DashboardViewModel.swift — computeStreak(), refresh()

computeStreak filters only **complete** logs, then checks if each date equals **startOfDay(now)**. If today's log is incomplete, the first matching date is yesterday — which doesn't equal *check*, so the loop breaks immediately and returns streak = 0. The guard if streak > 0 && todayLog?.isComplete != true in refresh() is therefore never true, so scheduleStreakAtRisk() never fires. The entire streak-at-risk notification flow

is silently broken.

**Suggested fix:**

```
var check = Calendar.current.startOfDay(for: .now)
if todayLog?.isComplete != true {
check = Calendar.current.date(byAdding: .day, value: -1, to: check)!
}
```

#  
2

**stressFatiguePattern checks the wrong log index**

High

Services/PatternEngine.swift — stressFatiguePattern(), line 72

When stress breaks at index *i*, logs[i] is the first normal-stress day. The detector then checks logs[i + 1] for fatigue — that is *two days after the streak ends*, not the recovery day itself. The check should be on logs[i].

**Suggested fix:**

```
// wrong:
let nextSymptoms = logs[i + 1].symptoms.map { $0.name.lowercased() }
// correct:
let nextSymptoms = logs[i].symptoms.map { $0.name.lowercased() }
```

#  
3

**Silent save failures give false positive feedback**

High

Features/DailyLog/DailyLogViewModel.swift:29 · Features/WeeklyReview/WeeklyReviewViewModel.swift:47

Both use try? context.save(). If the save fails the user receives a haptic success response and sees the 'Log complete!' screen, while their data was silently discarded. At minimum propagate the error to the caller; ideally present an alert so the user knows their entry was not saved.

#  
4

**PatternEngine.analyze() is a stub — misleading dead code**

Medium

Services/PatternEngine.swift:26-29 · Features/DailyLog/DailyLogViewModel.swift:33

analyze() is called after every log save but its body is empty. The comment 'In production: persist computed insights to SwiftData' implies future work, but in its current form it accepts a ModelContext it never uses and implies side effects that don't happen. Either implement persistence or remove the method and the call-site in DailyLogViewModel.save().

#  
5

**Streak-at-risk notification fires the wrong day after 9 PM**

Medium

Services/NotificationService.swift:60-68

UNCalendarNotificationTrigger with past time components automatically fires on the **next** matching time. If a user opens the app at 10 PM, the notification is scheduled for 9 PM the following day — 3 hours late. Add an early-return guard when the current hour is already past 21:00.

**Suggested fix:**

```
let hour = Calendar.current.component(.hour, from: .now)
guard hour < 21 else { return }
```

#  
6**Default sleepHours = 7.0 skews sleep correlation analysis**

medium

Models/DailyLog.swift:37 · Services/PatternEngine.swift — moodSleepCorrelation()

DailyLog initialises sleepHours to 7.0. If a user never touches the sleep slider the log records 7.0 hours, and moodSleepCorrelation's filter `$0.sleepHours > 0` won't exclude those logs. This inflates the population and biases the average. The `didEditMetrics` flag already exists — use it to exclude unedited logs from sleep analysis.

**Suggested fix:**

```
let pairs = logs.filter { $0.sleepHours > 0 && $0.didEditMetrics }
```

**Code Quality**#  
7**InsightCard.id is var instead of let**

Low

Features/Insights/InsightsViewModel.swift:6

Mutable identifiers break Identifiable semantics — SwiftUI diff-ing can produce unexpected re-renders if an id changes. Change to `let id = UUID()`.

#  
8**Hardcoded magic confidence scores**

Low

Services/PatternEngine.swift — energyTrendDecline():107, moodSleepCorrelation():127

`energyTrendDecline` returns confidence: 0.85 and `moodSleepCorrelation` returns confidence: 0.78 unconditionally regardless of the data. The other two detectors compute confidence from actual occurrence rates. Either derive the value from the data or remove the field from those cards so it isn't surfaced as a meaningful number in the UI.

#  
9**Force unwraps on Calendar.date(byAdding:)**

Low

Features/Dashboard/DashboardViewModel.swift:33 · Services/HealthKitService.swift:29-31

`Calendar.date(byAdding:to:)` returns an Optional. Both call sites force-unwrap with `!`. While calendar arithmetic rarely fails in practice, use guard let with a graceful fallback to avoid any edge-case crash.

#  
10**Synchronous pattern computation on @MainActor**

Low

Services/PatternEngine.swift — allInsights(from:)

All four pattern detectors iterate over the full log array synchronously on the main thread. With moderate data (365+ logs) this is fast, but the approach won't scale. Consider wrapping computation in `Task.detached` or moving `PatternEngine` to a background actor.

#  
11**PDF export overwrites previous file**

Low

PDFBuilder writes to a fixed filename in the temp directory (cadence-doctor-report.pdf). Generating two reports quickly can overwrite the first before the share sheet reads it. Append a timestamp or UUID to the filename.

**Suggested fix:**

```
let stamp = Int(Date.now.timeIntervalSince1970)
let filename = type == .doctor
? "cadence-doctor-report-\(stamp).pdf"
: "cadence-personal-summary-\(stamp).pdf"
```